

Assignment 4– Procedure and Function

Name: Aryan Ligade

Prn-124b1f155

class/div-Sy(A4)

Case Study: Airline Management System

1. Aim

To understand and implement SQL Stored Procedures, Functions, and Triggers for automating database operations, performing complex calculations, and maintaining data integrity in a relational database system.

2. Problem Statement (Case Study)

a. Implement at least 4 Stored Procedures for INSERT, UPDATE, DELETE, and SELECT operations. b. Create at least 4 User-Defined Functions to perform calculations and data retrieval. c. Implement BEFORE and AFTER Triggers to maintain data consistency and logging.

3. Database Design

Entity: Airplane This entity stores information about airplanes in the system.

- plane_id → Integer, Primary Key, Auto Increment
- plane_name → String (Unique) • capacity → Integer (Number of seats) • ticket_price → Decimal (Price per ticket)
- total_amount → Decimal (capacity × ticket price)

4. Database Schema

```
Command Prompt - mysql -u X + v
C:\Users\ASUS>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> -- Step 1: Create Database
mysql> CREATE DATABASE AirplaneDB;
Query OK, 1 row affected (0.03 sec)

mysql>
mysql> -- Step 2: Use Database
mysql> USE AirplaneDB;
Database changed
mysql>
mysql> -- Step 3: Create Table
mysql> CREATE TABLE Airplane (
->     plane_id INT PRIMARY KEY AUTO_INCREMENT,
->     plane_name VARCHAR(50),
->     capacity INT,
->     ticket_price DECIMAL(10,2)
-> );
```

5. Insert Sample Data

```
mysql>
mysql> -- Step 4: Insert Sample Data
mysql> INSERT INTO Airplane (plane_name, capacity, ticket_price)
-> VALUES
-> ('Indigo A320', 180, 5000),
-> ('Air India B737', 150, 6500),
-> ('SpiceJet Q400', 90, 4000);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

6. STORED PROCEDURES

6.1 Procedure 1: Add Airplane (INSERT)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE AddAirplane(
  ->     IN p_name VARCHAR(50),
  ->     IN p_capacity INT,
  ->     IN p_price DECIMAL(10,2)
  -> )
  -> BEGIN
  ->     INSERT INTO Airplane(plane_name, capacity, ticket_price)
  ->     VALUES(p_name, p_capacity, p_price);
  -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Call
mysql> CALL AddAirplane('Vistara A321', 200, 7000);
```

6.2 Procedure 2: Update Ticket Price

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE UpdatePrice(
  ->     IN p_id INT,
  ->     IN new_price DECIMAL(10,2)
  -> )
  -> BEGIN
  ->     UPDATE Airplane
  ->     SET ticket_price = new_price
  ->     WHERE plane_id = p_id;
  -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Call
mysql> CALL UpdatePrice(1, 5500);
Query OK, 1 row affected (0.01 sec)
```

6.3 Procedure 3: Delete Airplane

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE DeleteAirplane(
    ->     IN p_id INT
    -> )
    -> BEGIN
    ->     DELETE FROM Airplane
    ->     WHERE plane_id = p_id;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Call
mysql> CALL DeleteAirplane(3);
Query OK, 1 row affected (0.00 sec)
```

6.4 Procedure 4: Display All Airplanes (SELECT)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE ShowAirplanes()
    -> BEGIN
    ->     SELECT * FROM Airplane;
    -> END $$
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Call
mysql> CALL ShowAirplanes();
+-----+-----+-----+-----+
| plane_id | plane_name | capacity | ticket_price |
+-----+-----+-----+-----+
| 1 | Indigo A320 | 180 | 5500.00 |
| 2 | Air India B737 | 150 | 6500.00 |
| 4 | Vistara A321 | 200 | 7000.00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER $$
mysql>
```

7. USER-DEFINED FUNCTIONS

7.1 Function 1: Get Revenue (capacity × ticket_price)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION GetRevenue(p_id INT)
  -> RETURNS DECIMAL(10,2)
  -> DETERMINISTIC
  -> BEGIN
  ->     DECLARE total DECIMAL(10,2);
  ->
  ->     SELECT capacity * ticket_price INTO total
  ->     FROM Airplane
  ->     WHERE plane_id = p_id;
  ->
  ->     RETURN total;
  -> END $$
```

Query OK, 0 rows affected (0.01 sec)

```
mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Use
mysql> SELECT GetRevenue(1);
```

GetRevenue(1)
990000.00

7.2 Function 2: Get Capacity

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION GetCapacity(p_id INT)
  -> RETURNS INT
  -> DETERMINISTIC
  -> BEGIN
  ->     DECLARE cap INT;
  ->
  ->     SELECT capacity INTO cap
  ->     FROM Airplane
  ->     WHERE plane_id = p_id;
  ->
  ->     RETURN cap;
  -> END $$
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Use
mysql> SELECT GetCapacity(2);
+-----+
| GetCapacity(2) |
+-----+
|             150 |
+-----+
1 row in set (0.00 sec)
```

7.3 Function 3: Get Ticket Price

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION GetPrice(p_id INT)
  -> RETURNS DECIMAL(10,2)
  -> DETERMINISTIC
  -> BEGIN
  ->     DECLARE price DECIMAL(10,2);
  ->
  ->     SELECT ticket_price INTO price
  ->     FROM Airplane
  ->     WHERE plane_id = p_id;
  ->
  ->     RETURN price;
  -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Use
mysql> SELECT GetPrice(1);
+-----+
| GetPrice(1) |
+-----+
|       5500.00 |
+-----+
1 row in set (0.00 sec)
```

7.4 Function 4: Count Total Airplanes

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION TotalPlanes()
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->
    ->     SELECT COUNT(*) INTO total FROM Airplane;
    ->
    ->     RETURN total;
    -> END $$
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Use
mysql> SELECT TotalPlanes();
+-----+
| TotalPlanes() |
+-----+
|              3 |
+-----+
1 row in set (0.01 sec)
```

8. TRIGGERS IMPLEMENTATION

8.1 Modify Table - Add total_amount Column

```
mysql> -- Add column first
mysql> ALTER TABLE Airplane
    -> ADD total_amount DECIMAL(12,2);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

8.2 Trigger 1: BEFORE INSERT - Calculate Total Amount

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_insert_airplane
  -> BEFORE INSERT ON Airplane
  -> FOR EACH ROW
  -> BEGIN
  ->     SET NEW.total_amount = NEW.capacity * NEW.ticket_price;
  -> END $$
Query OK, 0 rows affected (0.01 sec)
```

8.3 Trigger 2: BEFORE UPDATE - Recalculate Total Amount

```
mysql>
mysql> DELIMITER ;
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_update_airplane
  -> BEFORE UPDATE ON Airplane
  -> FOR EACH ROW
  -> BEGIN
  ->     SET NEW.total_amount = NEW.capacity * NEW.ticket_price;
  -> END $$
Query OK, 0 rows affected (0.01 sec)
```

8.4 Trigger 3: AFTER DELETE Maintain Log


```

mysql>
mysql> DELIMITER ;
mysql> -- Log Table
mysql> CREATE TABLE Airplane_Log (
    ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     message VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER after_delete_airplane
    -> AFTER DELETE ON Airplane
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Airplane_Log(message)
    ->     VALUES (CONCAT('Deleted airplane: ', OLD.plane_name));
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;

```

9. Testing & Outputs

9.1 Test INSERT Procedure and BEFORE INSERT Trigger

```

mysql>
mysql> DELIMITER ;
mysql> -- Insert (trigger works)
mysql> INSERT INTO Airplane (plane_name, capacity, ticket_price)
    -> VALUES ('Akasa A320', 180, 6000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- Update (trigger works)
mysql> UPDATE Airplane
    -> SET ticket_price = 7000
    -> WHERE plane_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- View data
mysql> SELECT * FROM Airplane;

```

plane_id	plane_name	capacity	ticket_price	total_amount
1	Indigo A320	180	7000.00	1260000.00
2	Air India B737	150	6500.00	NULL
4	Vistara A321	200	7000.00	NULL
5	Akasa A320	180	6000.00	1080000.00

```

4 rows in set (0.00 sec)

```

9.2 Test DELETE Procedure and AFTER DELETE Trigger

```
mysql>
mysql> -- Delete (trigger logs)
mysql> DELETE FROM Airplane WHERE plane_id = 2;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- Check log
mysql> SELECT * FROM Airplane_Log;
+-----+-----+
| log_id | message                                     |
+-----+-----+
|      1 | Deleted airplane: Air India B737         |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

10. Conclusion

Stored Procedures, User-Defined Functions, and Triggers are essential components of modern database management systems that enhance functionality, security, and performance. Key Learnings:

- **Stored Procedures:** Enable encapsulation of business logic within the database, reducing code redundancy and improving maintainability. They support various operations like INSERT, UPDATE, DELETE, and SELECT.
- **Functions:** Return single values and are useful for calculations and data transformations. They can be used directly in SELECT statements, making queries more readable and efficient.
- **Triggers:** Automatically execute code in response to database events, ensuring data integrity and consistency. BEFORE triggers validate/modify data, while AFTER triggers maintain logs and audit trails.

Benefits: ✓ Code reusability and maintainability ✓ Improved security through controlled access ✓ Enhanced performance with reduced network traffic ✓ Automatic enforcement of business rules ✓ Better data integrity and consistency

This practical demonstrates how these advanced SQL features work together to create a robust, efficient, and secure database management system for real-world applications.